

Bolide 从入门到精通

Bolide Team

目录

Bolide 从入门到精通	4
目录	5
第 1 章 认识 Bolide	6
1.1 Bolide 是什么	6
1.2 两种运行方式	6
1.3 设计取向与特性地图 (0.14.1)	7
1.4 适合做什么	7
1.5 最小程序	7
1.6 与其他语言的直觉对照	8
第 2 章 安装、工具链与项目结构	8
2.1 从源码构建	8
2.2 使用 Release 包	8
2.3 环境变量	8
2.4 常用 CLI	9
2.5 项目形态	9
2.6 仓库内部结构 (贡献 / 深读)	9
第 3 章 第一个程序与开发环境	10
3.1 Hello 进阶	10
3.2 用户输入	10
3.3 VS Code 插件	10
3.4 源码级错误体验	10
3.5 建议的学习节奏	11
第 4 章 基础语法: 变量、类型、表达式	11
4.1 let 与 var	11
4.2 基本类型	11
4.3 f-string	12
4.4 类型转换	12
4.5 表达式与运算符	12
4.6 元组与解构	13
4.7 if let / while let	13
第 5 章 控制流、作用域与求值顺序	13
5.1 if / elif / else	13
5.2 while	14

5.3 for	14
5.4 break / continue	14
5.5 作用域	14
5.6 求值顺序提示	15
第 6 章 函数、参数、泛型与内联	15
6.1 定义与调用	15
6.2 默认参数、具名、变长	15
6.3 参数传递模式: 默认 / ref / owned	16
6.4 返回值生命周期 from (跳过 ARC)	18
6.5 泛型函数	19
6.6 inline fn	20
6.7 重载	20
6.8 与内置名隔离	20
第 7 章 一等函数、闭包与高阶编程	20
7.1 函数是值	20
7.2 闭包	21
7.3 map / filter	21
第 8 章 字符串、列表、字典、元组与切片	22
8.1 字符串方法	22
8.2 切片	22
8.3 列表	23
8.4 字典	23
8.5 元组	24
第 9 章 值类型 value	24
第 10 章 类、对象、继承与多继承	24
10.1 定义与构造	24
10.2 继承与 super	25
10.3 多继承 (安全子集)	25
第 11 章 Trait、dyn Trait 与协议自动满足	26
11.1 trait / impl	26
11.2 泛型约束	26
11.3 dyn Trait (运行时多态)	26
11.4 Supertrait	27
11.5 协议自动满足	27
第 12 章 运算符重载	27
第 13 章 生成器与 yield	28
第 14 章 宏系统	29
14.1 心智模型	29
14.2 内置与自定义	29
14.3 导出与导入	30
14.4 属性宏与 derive	30
14.5 重复与 comptime	30

第 15 章 装饰器与上下文管理器 with	31
15.1 运行时装饰器	31
15.2 with	32
第 16 章 枚举、模式匹配与错误处理	32
16.1 异常 try / catch / throw / finally	32
16.2 Result / Option 与 ?	33
16.3 ! 与 try 表达式	33
16.4 实践建议	34
第 17 章 模块系统与包管理器	34
17.1 文件模块	34
17.2 路径解析顺序	34
17.3 bolide.toml 与命令	35
第 18 章 类型系统深入	35
18.1 静态类型 + 推断	35
18.2 类型一览	36
18.3 dynamic 的边界	36
18.4 函数类型与 C ABI	36
第 19 章 内存管理: ARC、from、weak、unowned	36
19.1 ARC 默认模型	36
19.2 传参如何影响 ARC	37
19.3 from: 返回借用并跳过 ARC	37
19.4 weak / unowned	39
19.5 概念总表	39
19.6 并发与共享	40
19.7 调试与学习建议	40
第 20 章 并发: 线程、通道、async/await	40
20.1 概念对照	40
20.2 async / await	41
20.3 spawn / pool	41
20.4 spawn all / select	41
20.5 通道与 select	41
20.6 原子与同步	42
第 21 章 FFI 与 AOT / LLVM 发布	42
21.1 Bolide 调 C	42
21.2 C 调 Bolide	43
21.3 AOT 发布清单	43
21.4 性能后端选择	43
第 22 章 标准库全景	44
22.1 导入约定	44
22.2 模块地图	44
22.3 选型速查	44
22.4 失败语义	44
22.5 实现分层 (进阶)	44

22.6 核心模块用法精讲	45
第 23 章 Web、模板与数据库	46
23.1 最小 Web 服务	46
23.2 HTTP 客户端	47
23.3 模板引擎	48
23.4 文件数据库 std/db	48
23.5 SQLite	49
23.6 把三块拼成应用	49
第 24 章 GUI 开发	49
24.1 模型	49
24.2 最小窗口	49
24.3 布局 API	50
24.4 常用控件	50
24.5 示例与发布	50
第 25 章 报错诊断、调试与测试	51
25.1 读懂编译错误	51
25.2 调试手段	51
25.3 轻量测试	51
25.4 常见坑	52
第 26 章 性能优化与工程实践	52
26.1 优化优先级	52
26.2 安全与速度的边界	52
26.3 工程约定	53
26.4 基准与对照	53
26.5 发布检查清单	53
第 27 章 综合项目实战	54
27.1 项目 A: 命令行任务管理器 (可落地骨架)	54
27.2 项目 B: 迷你博客	54
27.3 项目 C: GUI 计算器	55
27.4 项目 D: 数值小引擎	55
27.5 项目 E: 导出 C 库	55
27.6 项目 F: 特性总览程序	55
第 28 章 附录	56
附录 A ·语法速查	56
附录 B ·命令速查	57
附录 C ·学习资源	57
附录 D ·版本与许可	57
结束语	57

Bolide 从入门到精通

版本基准: Bolide 0.14.1

适用平台: Windows / Linux

许可: MIT

本书面向第一次接触 Bolide 的读者, 覆盖 AOT 发布、FFI、并发、内存模型、宏与装饰器、Trait、标准库与工程化实践。
让您能从写出第一个 print 到独立交付 CLI、Web、GUI 与原生库。

目录

第一部分•入门

1. 认识 Bolide
2. 安装、工具链与项目结构
3. 第一个程序与开发环境
4. 基础语法: 变量、类型、表达式
5. 控制流、作用域与求值顺序

第二部分•语言核心

6. 函数、参数 (含 ref/owned)、泛型与内联
7. 一等函数、闭包与高阶编程
8. 字符串、列表、字典、元组与切片
9. 值类型 value
10. 类、对象、继承与多继承
11. Trait、dyn Trait 与协议自动满足
12. 运算符重载
13. 生成器与 yield
14. 宏系统
15. 装饰器与上下文管理器 with
16. 枚举、模式匹配与错误处理

第三部分•系统与工程

17. 模块系统与包管理器
18. 类型系统深入

- 19. 内存管理：ARC、from、weak、unowned
 - 20. 并发：线程、通道、async/await
 - 21. FFI 与 AOT / LLVM 发布
 - 22. 标准库全景
 - 23. Web、模板与数据库
 - 24. GUI 开发
 - 25. 报错诊断、调试与测试
 - 26. 性能优化与工程实践
 - 27. 综合项目实战
 - 28. 附录：语法速查·命令速查·路线图
-

第 1 章 认识 Bolide

1.1 Bolide 是什么

Bolide 是一门现代编程语言：语法偏向可读、接近 Python 的表达习惯，执行模型却走原生编译路线——基于 **Cranelfit** 做 JIT（即时编译）与 AOT（提前编译），并可选用 **LLVM** 后端追求接近 C 的峰值性能。

一句话概括：

写起来像脚本语言，跑起来像编译语言，发得出去是单个原生可执行文件。

源文件扩展名一般为 .bl。

1.2 两种运行方式

方式	命令	场景
JIT	<code>bolide run main.bl</code>	开发、调试、快速试验
AOT	<code>bolide compile main.bl</code> <code>-o main</code>	发布、部署、分发独立可执行文件

AOT 产物可脱离 Bolide 解释器/JIT 单独运行；Windows 上通常得到 .exe，Linux/macOS 得到无后缀可执行文件。

可选后端：

```
# 默认 Cranelift
bolide run app.bl
bolide compile app.bl -o app

# 可选 LLVM (需本机 clang; Windows 还需 Lld-Link)
bolide run app.bl --backend llvm
bolide compile app.bl -o app_llvm.exe --backend llvm
```

参考基准 (仓库 bench/, 同一台 Windows 机、best of 3): LLVM 几何平均约 $1.03 \times C-03$, Cranelift 约 $1.3-1.6 \times$ (随算法与边界检查而变)。list[i] 始终带边界检查——Bolide 不以牺牲安全换速度。

1.3 设计取向与特性地图 (0.14.1)

类别	能力
表达	let/var、f-string、切片、默认/具名/*args/**kwargs
函数	一等函数、闭包、map/filter、泛型、inline fn
数据	list/dict/tuple、value 值类型、bigint/decimal
OOP	class、继承、安全子集多继承、运算符重载
抽象	trait/impl、T: Trait、dyn Trait、协议自动满足
元编程	声明式宏 name!、@derive、属性宏、comptime
Python 风格	@装饰器、with 上下文管理器、yield 生成器
并发	spawn/pool、channel、select、async/await
系统	双向 FFI、AOT 可执行文件 / 静态库 + C 头文件
应用库	Web / HTTP / GUI / JSON / DB / CLI / 正则 / 模板...
安全	ARC (原子 RC)、from 借用、weak/unowned 确定性 trap
体验	源码级错误 (行号、caret、help)、VS Code 插件

1.4 适合做什么

- 性能敏感的 **命令行工具**
- 需要 **单文件原生部署** 的服务端 / Web
- 需要 **C 互操作** 的边界代码
- 带 **GUI** 的小工具
- 用高级语法组织的中大型业务逻辑

1.5 最小程序

```
print("hello Bolide");

bolide run hello.bl
bolide compile hello.bl -o hello
./hello # Windows: hello.exe
```

1.6 与其他语言的直觉对照

若你熟悉...	Bolide 里像什么
Python	切片、默认参数、*args、装饰器、with、yield、方法调用风格
Rust	let 默认不可变、Result/Option、?、trait、ARC/weak
Go	channel、select、goroutine 风格的 spawn/pool
C/C++	AOT 原生二进制、export fn、FFI
TypeScript	可选类型标注 + 推断、func(...) 函数类型

第 2 章 安装、工具链与项目结构

2.1 从源码构建

需要 **Rust 工具链** (cargo)。

```
git clone https://github.com/streetartist/bolide.git
cd bolide
cargo build --release
```

可执行文件一般在：

- target/release/bolide (或 Windows 上的 bolide.exe)
- 仓库根目录也可能有已构建的 bolide.exe (视发布方式而定)

运行示例：

```
cargo run --release -- run examples/hello.bl
cargo run --release -- run examples/neon_lang.bl
cargo run --release -- run examples/starfield.bl
```

2.2 使用 Release 包

从 Release 下载对应平台包后：

```
# Windows
bolide.exe run your_program.bl

# Linux / macOS
./bolide run your_program.bl
```

发行布局中，std/ 通常与可执行文件同级，便于 import "std/fs" 解析。

2.3 环境变量

变量	作用
BOLIDE_HOME	开发期指向仓库根，便于解析 std/...
BOLIDE_CACHE_DIR	包缓存目录；默认 Windows 为 %LOCALAPPDATA%\bolide, Unix 为 ~/.cache/bolide

2.4 常用 CLI

```

bolide run src/main.bl           # JIT 运行
bolide compile src/main.bl -o app # AOT 可执行文件
bolide compile lib.bl --lib --header # 静态库 + C 头
bolide expand file.bl           # 查看宏/装饰器展开 (调试)
bolide new my_app               # 创建项目骨架
bolide add dep@1.0.0            # 添加依赖
bolide install                   # 解析依赖, 生成 Lock

```

后端选择:

```

bolide run app.bl --backend cranelift # 默认
bolide run app.bl --backend llvm
bolide compile app.bl -o app --backend llvm

```

2.5 项目形态

单文件:

```
hello.bl
```

带包管理:

```

my_app/
  bolide.toml
  src/
    main.bl
    util.bl

```

`bolide run / bolide compile` 会**向上查找** `bolide.toml`, 解析依赖后注入编译器。

2.6 仓库内部结构 (贡献 / 深读)

```

bolide/
├── crates/
│   ├── bolide-cli/           # 命令行入口
│   ├── bolide-parser/        # PEG 语法 (pest)
│   ├── bolide-compiler/      # Cranelift / LLVM 后端
│   ├── bolide-runtime/       # 运行时 (字符串、列表、Web、GUI...)
│   └── bolide-pkg/           # 包管理
└── std/                      # 标准库 .bl 包装层

```

└─ examples/	# 示例
└─ tests/	# 回归测试
└─ bench/	# 性能基准
└─ vscode-bolide/	# VS Code 插件
└─ docs/	# 文档与本书

第 3 章 第一个程序与开发环境

3.1 Hello 进阶

```
let name: str = "Bolide";
print("hello " + name);
print(f"hello {name}");
```

类型注解可写可省：

```
let a: int = 10;
let b = 20;
print(a + b);
```

3.2 用户输入

```
let name: str = input("name: ");
print("hello " + name);

let raw: str = input("age: ");
let age: int = int(raw);
print(age + 1);
```

`input()` 返回 `str`；需要数字时用 `int()` / `float()` 转换。

3.3 VS Code 插件

仓库提供 `vscode-bolide`：

1. 将文件夹复制到 VS Code 扩展目录，或打包 VSIX 安装。
2. 设置 `bolide.executablePath` 指向本机 `bolide`。
3. 打开 `.bl` 文件，`Ctrl+Shift+R` 运行当前文件。

3.4 源码级错误体验

```
let x = missing_name + 1;
print(x);
```

编译器会给出文件、行号、caret 与 help，例如提示未定义名、拼写或 import 问题。
bolide run、bolide compile 与 REPL 均走同一套诊断。

3.5 建议的学习节奏

阶段	章节	目标
第 1 周	1-8	能写脚本、处理列表字符串
第 2 周	9-16	OOP、trait、宏、错误模型
第 3 周	17-21	包、内存、并发、AOT/FFI
第 4 周	22-27	标准库与完整小项目

第 4 章 基础语法：变量、类型、表达式

4.1 let 与 var

关键字	含义
let	不可变绑定 ：不能重新赋值；也不能通过该绑定做 <code>xs[i]=...</code> 、 <code>push</code> 等原地修改
var	
	可变绑定 ：允许赋值、复合赋值、容器原地修改

```
let x: int = 42;
// x = 1;  // 错误
```

```
var n: int = 1;
n = n + 1;
n += 10;
```

```
var items: list<int> = [1, 2];
items.push(3);
```

顶层声明是**全局变量**；函数内同名 let/var 会**遮蔽**全局。

4.2 基本类型

类型	含义	示例
int	64 位整数	42、1_000_000
float	双精度浮点	3.14
bool	布尔	true / false
str	字符串	"hi"、f-string
bigint	任意精度整数	123b / 123B
decimal	十进制定点	3.14d / 3.14D

类型	含义	示例
dynamic	动态值	异构数据边界
none / null	空值（语境相关）	—

```
let x: int = 42;
let pi: float = 3.14159;
let name: str = "Bolide";
let flag: bool = true;
let big: bigint = 123456789012345678901234567890b;
let precise: decimal = 3.14159265358979d;
let million: int = 1_000_000;
```

字符串转义: `\ " \\ \n \t \r \0`。

4.3 f-string

```
let id: int = 42;
let name: str = "Bolide";
print(f"user={name} id={id}");
print(f"sum={1 + 2}");
print(f"brace={{ok}}");           // 字面量花括号
print(f"nested={f'inner'}");
```

`{expr}` 求值后转成字符串拼接; `{{ / }}` 表示字面量 `{ / }`。

4.4 类型转换

```
let a: int = int(3.7);           // 截断 → 3
let b: int = int("123");
let e: float = float(100);
let h: str = str(12345);
let m: bigint = bigint(100);
let n: decimal = decimal(3.14);
```

布尔在字符串/列表等 API 中常表现为 `true/false` (`str(...)` 输出亦然)。

4.5 表达式与运算符

- 算术: `+` `-` `*` `/` `%`
- 比较: `==` `!=` `<` `<=` `>` `>=` (**不支持**链式比较 `a < b < c`, 会语法错误)
- 逻辑: `and` / `or` / `not` (以及一元 `!`)
- 复合赋值: `+=` `-=` `*=` `/=` `%=`
- 短路: `&&` / `||` **不可**被运算符重载

4.6 元组与解构

```
let t = (10, 20, 30);
let (a, b, c) = t;
let (x, _, z) = t;      // _ 丢弃
_ = a + b;              // 求值但丢弃结果

class Point {
  x: int;
  y: int;
}
let p: Point = Point(3, 4);
let Point { x, y } = p;
let Point { x: px, y: _ } = p;
```

4.7 if let / while let

```
fn find(flag: int) -> Option<int> {
  if flag > 0 { return Option.Some(flag); }
  return Option.None();
}

if let Option.Some(v) = find(7) {
  print(v);
} else {
  print(0);
}

var i: int = 1;
fn take(n: int) -> Option<int> {
  if n <= 3 { return Option.Some(n); }
  return Option.None();
}
while let Option.Some(v) = take(i) {
  print(v);
  i += 1;
}
```

语义上脱糖为 match。

第 5 章 控制流、作用域与求值顺序

5.1 if / elif / else

```
if x > 0 {
  print("positive");
}
```

```

} elif x < 0 {
    print("negative");
} else {
    print("zero");
}

```

5.2 while

```

var x: int = 5;
while x > 0 {
    x = x - 1;
}

```

5.3 for

```

for i in range(5) { print(i); }           // 0..4
for i in range(3, 7) { print(i); }       // 3..6
for i in range(0, 10, 2) { print(i); }
for i in range(10, 0, -2) { print(i); }  // 负步长

let nums: list<int> = [10, 20, 30];
for n in nums { print(n); }

let scores = {"Alice": 100, "Bob": 85};
for k, v in scores {
    print(k);
    print(v);
}

```

任意实现 `next() -> Option<T>` 的对象（含生成器）都可 `for` 遍历。

5.4 break / continue

```

var total: int = 0;
for i in range(10) {
    if i == 3 { continue; }
    if i == 6 { break; }
    total += i;
}

```

`continue` 会执行循环步进；提前跳出时正确释放作用域内 `RC` 变量。

5.5 作用域

- 块 `{ ... }` 引入局部作用域。

- 函数参数是局部绑定。
- 函数内声明遮蔽同名全局。
- 全局与局部能力一致：可作 ref 实参、通道等。

5.6 求值顺序提示

- 短路逻辑先求左再可能求右。
 - `await a + b` 解析为 `(await a) + b`。
 - 函数实参一般按书写顺序求值。
 - 不要依赖未定义的副作用顺序做“技巧代码”。
-

第 6 章 函数、参数、泛型与内联

6.1 定义与调用

```
fn add(a: int, b: int) -> int {
    return a + b;
}

fn greet(name: str = "world", punctuation: str = "!") {
    print("hello " + name + punctuation);
}

greet();
greet(name="Bolide");
greet(punctuation="?", name="B");
```

6.2 默认参数、具名、变长

```
fn total(base: int = 10, *nums: int, **opts: int) -> int {
    var sum: int = base;
    for n in nums {           // list<int>
        sum += n;
    }
    if opts.contains("bonus") { // dict<str, int>
        sum += opts["bonus"];
    }
    return sum;
}
```

```
let xs: list<int> = [2, 3];
let kwargs: dict<str, int> = {"bonus": 4};
print(total());
print(total(1, *xs, **kwargs));
print(total(base=5, bonus=7));
```

规则摘要：

- name: T = expr: 默认值
- *args: T: 多余位置参数 → list<T>
- **kwargs: T: 多余具名参数 → dict<str, T>
- 调用侧: name=value / name: value、*list、**dict
- *args 在普通参数后; **kwargs 必须最后
- 无对应形参且无 **kwargs 时, 未知具名参数编译报错

6.3 参数传递模式：默认 / ref / owned

Bolide 对**可能走引用计数的类型** (如 bigint、str、list、对象等) 提供三种传参模式。
关键字写在**参数名前面** (与 from 写在签名末尾不同)。

模式	写法	调用方	被调用方	RC / 开销
默认 (借用/按值语义视类型)	fn f(x: T)	调用后仍可用	通常只读使用	对 ARC 对象: 常传指针、 不额外 retain (函数内只读)
ref (可写引用)	fn f(ref x: T)	传入变量可被改	可改调用方变量	传「变量地址」; 适合原地修改
owned (所有权转移)	fn f(owned x: T)	失去该值 , 再用不合法	负责用完/释放	移动语义, 避免双份 RC

默认模式 (Borrow)

适合只读查看、计算, 不拿走所有权:

```
fn print_bigint(x: bigint) {
    print(x);
}

fn add_bigints(a: bigint, b: bigint) -> bigint {
    return a + b;    // 返回新值, 通常会走正常构造/RC
}

let a: bigint = 100B;
```



```
let b: bigint = 200B;
print_bigint(a);
let sum: bigint = add_bigints(a, b);
print(a);           // 仍可用
print(sum);
```

对标量 int/float/bool，默认就是按值拷贝，一般不必纠结 RC。

ref: 修改调用方变量

调用时**直接写变量名** (不必写 ref 关键字); 形参声明 ref 即可。

```
fn double_value(ref x: bigint) {
    x = x + x;
}

fn swap_values(ref a: bigint, ref b: bigint) {
    let temp: bigint = a;
    a = b;
    b = temp;
}

fn increment(ref n: bigint) {
    n = n + 1B;
}

var val: bigint = 10B;
double_value(val);    // val 变成 20B

var m: bigint = 111B;
var n: bigint = 222B;
swap_values(m, n);    // 交换

var counter: bigint = 0B;
increment(counter);
increment(counter);
```

ref 也可用于 float 等:

```
fn add_to(x: float, ref out: float) {
    out = out + x;
}
```

全局变量与局部变量都可作 ref 实参。

owned: 拿走所有权

```
fn consume_bigint(owned x: bigint) {
    print(x);
}
```

```

    // 函数结束时 x 被释放 (最后一个 owner)
}

var x: bigint = 500B;
consume_bigint(x);
// print(x); // 编译错误: x 已被 move, 不能再使用

```

适用场景：把大对象交给通道/后台任务、明确「调用方不再持有」、避免无意义的 retain/release 成对操作。

选型口诀

1. **只读用默认** —— 最常见，省事。
2. **要改调用方绑定用 ref** —— 类似「输出参数 / 原地修改」。
3. **明确移交生命周期用 owned** —— 调用后作废原变量。
4. **返回值想「借参数、不涨 RC」用 from** —— 见第 6.4 节与第 19 章（常与 ref 参数搭配）。

更完整的借用检查规则与 weak/unowned 见 **第 19 章**。

6.4 返回值生命周期 from (跳过 ARC)

普通返回 ARC 对象时，往往会 **retain 返回值**（调用方拿到强引用）。

若返回值其实只是**借用某个参数**、且保证不比参数活得更久，可用 from 声明依赖，**跳过这次 ARC 开销**：

```

// 返回值的生命周期依赖参数 x: b 借用 a, 不增加引用计数
fn get_value(ref x: bigint) -> bigint from x {
    return x;
}

let a: bigint = 100B;
let b: bigint = get_value(a); // 借用，非新的强引用拷贝
print(b);

```

语法位置：

```
fn 名 (参数列表) -> 返回类型 from 参数名 { ... }
```

- from 写在**签名末尾**（与参数前的 ref/owned 位置不同）。
- from 后跟**参数名**，表示返回值来源于该参数。
- 常见搭配：ref 参数 + from 同一参数（读借用再还回去）。

编译器会做严格检查（违反则**编译报错**）：

规则	含义
来源必须匹配	返回值必须来自 from 声明的那个参数
无悬空	来源离开作用域后，借用不可继续存活
来源冻结	借用存活期间， 禁止对来源变量重新赋值 （旧对象会被释放）
禁止逃逸	借用不能进 list/dict/字段/通道/spawn，也不能从 未声明 from 的函数再返回

需要逃逸时，先**显式拷贝**再存储，例如 `bigint(b)`、构造新 `str/list` 等，重新变成拥有所有权的值。

```
fn first_item(ref xs: list<str>) -> str from xs {
    return xs[0];
}

var names: list<str> = ["Ada", "Bob"];
let head: str = first_item(names); // 借用 names[0] 的生命周期
// names = []; // 若 head 仍存活，对 names 重赋值会被拒绝
print(head);
```

何时用 from:

- 热路径上反复「取字段 / 取元素 / 转发参数」又不想每次 `RC ±1`。
- API 语义本来就是「返回内部视图 / 借用」，调用约定清晰。

何时不要用:

- 返回值需要独立存活、存进容器、跨线程——用普通返回（带 `RC`）或显式拷贝。
- 逻辑上会产生新对象（拼接字符串、`a + b` 等）——与「借参数」不符，不要硬写 `from`。

与参数模式对照：

目标	写法
只读看一眼	默认参数
改调用方变量	<code>ref</code>
拿走并释放/移交	<code>owned</code>
返回借用、省 <code>retain</code>	<code>-> T from param</code> （常加 <code>ref param</code> ）

6.5 泛型函数

```
fn id<T>(x: T) -> T { return x; }
fn pair<T, U>(a: T, b: U) -> (T, U) { return (a, b); }
fn wrap<T>(x: T) -> list<T> { return [x]; }

print(id(42));
print(id("hello"));
print(pair(10, "x"));
```

调用时通常**无需**写类型实参；编译器单态化。

支持顶层泛型、**class 泛型方法**、以及**泛型函数作一等值**（需可推断的 `func(...)` 类型）：

```
let f: func(int) -> int = id;
print(f(1));

class Box {
    value: int;
    fn map<U>(f: func(int) -> U) -> U {
        return f(self.value);
    }
}
```

无注解时不能 `let f = id`; (无法确定实例)。

6.6 inline fn

```
value Vec3 { x: float; y: float; z: float; }

inline fn v3_add(a: Vec3, b: Vec3) -> Vec3 {
    return Vec3 { x: a.x + b.x, y: a.y + b.y, z: a.z + b.z };
}

inline fn sq(x: float) -> float {
    return x * x;
}
```

适合数值热路径上的短函数。编译器也会自动内联部分小叶子函数（标量参数/返回、单出口等）。

6.7 重载

可按参数类型重载函数；按实参类型匹配。类型失败后**不会**仅按参数个数盲目回退（0.14.x 修复了相关语义）。

6.8 与内置名隔离

运行时内部符号在 `@_` 命名空间，用户可用任意合法标识符（含 `print_bigint` 等）而不与内部 ABI 冲突。列表操作使用方法：`xs.push(3)`，而不是内部 `list_push`。

第 7 章 一等函数、闭包与高阶编程

7.1 函数是值

`fn` 用于声明/字面量；**函数值类型**统一写作 `func(T...) -> R`。

```
fn add1(x: int) -> int { return x + 1; }
fn double(x: int) -> int { return x * 2; }
```

```

let f = add1;
print(f(10));

let g: func(int) -> int = double;

fn apply(callback: func(int) -> int, x: int) -> int {
    return callback(x);
}
print(apply(double, 21));

fn pick(which: int) -> func(int) -> int {
    if which == 0 { return add1; }
    return double;
}

let fns: list<func(int) -> int> = [add1, double];
print(fns[0](5));

```

一等函数值（含列表中的函数、返回后再调用）走**闭包对象 ABI**：裸函数会自动 wrap。

7.2 闭包

```

let double: func(int) -> int = fn(x: int) -> int {
    return x * 2;
};

let n: int = 10;
let add_n = fn(x: int) -> int {
    return x + n;
};

fn make_adder(n: int) -> func(int) -> int {
    return fn(x: int) -> int {
        return x + n;
    };
}
let add5 = make_adder(5);
print(add5(10)); // 15

```

捕获的 ARC 对象会自动保活。

7.3 map / filter

```

fn double(x: int) -> int { return x * 2; }
fn is_even(x: int) -> bool { return x % 2 == 0; }
fn label(n: int) -> str { return "n=" + str(n); }

```

```
let nums: list<int> = [1, 2, 3, 4];
print(nums.map(double));
print(nums.filter(is_even));
print(nums.map(label));
```

注意：函数内类型推断完整；**顶层**调用建议显式标注结果类型。

第 8 章 字符串、列表、字典、元组与切片

8.1 字符串方法

```
let s: str = "Hello, World";
print(s.len());
print(s.upper());
print(s.lower());
print(s.contains("World"));
print(s.find("World"));
print(s.starts_with("Hell"));
print(s.ends_with("rld"));
print(s.replace("l", "L"));
print(s.count("l"));
print("  trim me  ".trim());
print("ab".repeat(3));
print(s.substring(0, 5));
print(s.char_at(1));
let parts: list<str> = "a,b,c".split(",");
```

别名：length/size、strip、index_of、includes、substr 等。

索引/切片按 **Unicode 码点**；len() 当前返回 **UTF-8 字节长度**（写跨语言文本时注意）。

8.2 切片

```
let text: str = "Hello, World";
print(text[0:5]);
print(text[7:]);
print(text[:5]);
print(text[:2]);
print(text[:-1]);
print(text[-1]);

let nums: list<int> = [10, 20, 30, 40, 50];
print(nums[1:4]);
print(nums[:-1]);

let t: (int, int, int, int) = (1, 2, 3, 4);
let mid: (int, int) = t[1:3];
```

语法: `seq[start:end:step]`, 均可省略; 负索引与负步长可用。

8.3 列表

边界检查: `list[i]` 始终检查; 越界读返回 `0/0.0`, 越界写忽略。预分配:

```
var flags: list<int> = [];  
flags.resize(1000, 0);  
flags.reserve(2000);
```

常用 API:

```
var nums: list<int> = [3, 1, 4, 1, 5, 9];  
nums.push(10);  
let x: int = nums.pop();  
print(nums.len());  
nums[0] = 100;  
nums.insert(1, 42);  
let removed: int = nums.remove(2);  
print(nums.contains(4));  
print(nums.index_of(4));  
print(nums.count(1));  
print(nums.first());  
print(nums.last());  
print(nums.is_empty());  
nums.reverse();  
nums.sort();  
nums.extend([100, 200]);  
let copy: list<int> = nums.copy();  
nums.clear();
```

列表拼接 `list + list` (含嵌套 `list<list<int>>`) 在 JIT/AOT 均正确支持。

8.4 字典

```
var scores: dict<str, int> = {"Alice": 100, "Bob": 90};  
print(scores["Alice"]);  
scores["Charlie"] = 95;  
scores.remove("Bob");  
print(scores.len());  
print(scores.contains("Alice"));  
print(scores.keys());  
print(scores.values());  
  
// 异构 → dict<dynamic, dynamic>  
let profile = {"name": "Bolide", 1: "Version", "active": true};
```

8.5 元组

```
let t: (int, str, bool) = (1, "a", true);
print(t[0]);
print(t[1:3]);
```

第 9 章 值类型 value

value 定义**轻量聚合**，适合 Vec2/Vec3、颜色、小型记录。按值构造与传递，字段用点号访问；JIT/AOT 均支持。

```
value Vec3 { x: float; y: float; z: float; }

let a: Vec3 = Vec3 { x: 1.0, y: 2.0, z: 3.0 };
let b: Vec3 = Vec3 { x: 4.0, y: 5.0, z: 6.0 };

fn dot(lhs: Vec3, rhs: Vec3) -> float {
    return lhs.x * rhs.x + lhs.y * rhs.y + lhs.z * rhs.z;
}

print(dot(a, b));
print(a.x);
```

与 class 的取舍：

	value	class
语义	按值聚合	引用语义 (ARC)
典型用途	数值、小记录	有身份、可变对象图、继承
开销	低 (拷贝字段)	堆分配 + RC

示例: examples/raytracer_vt.bl。

第 10 章 类、对象、继承与多继承

10.1 定义与构造

```
class Point {
    x: int;
    y: int;

    fn distance() -> int {
        return self.x * self.x + self.y * self.y;
    }
}
```



```

    fn move_by(dx: int, dy: int) {
        self.x = self.x + dx;
        self.y = self.y + dy;
    }
}

```

```

let p: Point = Point(3, 4);
print(p.distance());
p.move_by(1, 1);

```

构造函数按字段顺序初始化。

10.2 继承与 super

```

class Animal {
    age: int;
    fn get_age() -> int { return self.age; }
}

```

```

class Dog: Animal {
    name: int;
    fn bark() -> int { return 100; }
}

```

```

let dog: Dog = Dog(3, 42);
print(dog.get_age());
print(dog.bark());

```

方法分派按类型与重载实参匹配；super 沿主父类链调用。

10.3 多继承（安全子集）

```

class Child: Primary, Mixin1, Mixin2 { }

```

规则：

1. **第一个父类 (Primary)**：唯一参与**字段布局**与 super 链（可有字段）。
2. **其余父类 (Mixin)**：必须**无字段**；方法复制进子类。
3. 两个 mixin 同名方法且子类未覆盖 → **编译错误**（强制消歧）。

能力组合更推荐 **trait**；mixin 适合无状态工具类。

第 11 章 Trait、dyn Trait 与协议自动满足

11.1 trait / impl

```
trait Drawable {
    fn draw();
    fn label() -> str { return "shape"; } // 默认方法
}

class Circle {
    r: int;
}

impl Drawable for Circle {
    fn draw() { print(self.r); }
}

let c = Circle(3);
c.draw();
print(c.label());
```

无方法体 = 必须实现；带默认体可省略。

11.2 泛型约束

```
fn paint<T: Drawable>(x: T) {
    x.draw();
}

fn paint_count<T: Drawable + Countable>(x: T) {
    x.draw();
    print(x.count());
}
```

单态化时检查是否 impl；未实现则报错并提示。

11.3 dyn Trait (运行时多态)

```
fn paint(d: dyn Drawable) {
    d.draw();
}

paint(Circle(3));
let d: dyn Drawable = Circle(1);
d.draw();
```

编译期改写为合成类，运行时按 **class tag** 分派。注意 dyn 有词边界：dynamic 不会被拆成 dyn + amic。

11.4 Supertrait

```
trait Countable: Drawable {
  fn count() -> int;
}
// impl Countable 时须已具备 Drawable 能力
```

11.5 协议自动满足

类上若有对应方法，自动视为实现协议，可用于 T: Trait，无需手写 impl:

协议	方法
Add/Sub/Mul/Div/Mod	__add__ ...
Eq/Ord	__eq__ / __lt__
位运算相关	__and__ ...
Neg/Not	__neg__ / __not__
Iterator	next (通常 Option<T>)

任意带 next() 的 class 都可 for x in it { ... }。
标准定义见 std/traits。

第 12 章 运算符重载

在 class 上定义 Python 风格 dunder 方法。左操作数优先 left.__op__(right); 否则尝试右操作数反射 (__radd__ 等)。+= 等脱糖为 a = a + b。

运算符	方法	反射
+ - * / %	__add__ ...	__radd__ ...
== !=	__eq__ __ne__	对调
< <= > >=	__lt__ ...	对偶
& \ ^ << >>	__and__ ...	__rand__ ...
一元 - !/not	__neg__ __not__	—

```
class Vec {
  x: int;
  y: int;
  fn __add__(o: Vec) -> Vec {
    return Vec(self.x + o.x, self.y + o.y);
  }
  fn __radd__(n: int) -> int {
    return n + self.x + self.y;
  }
  fn __neg__() -> Vec {
```

```

        return Vec(0 - self.x, 0 - self.y);
    }
    fn __eq__(o: Vec) -> bool {
        return self.x == o.x && self.y == o.y;
    }
}
print(Vec(1, 2) + Vec(3, 4));
print(10 + Vec(1, 2));
print(-Vec(1, 2));

```

&& / || 不支持重载 (保持短路)。

第 13 章 生成器与 yield

含 yield 的函数是**生成器**：返回迭代器，按需 next()，或 for 遍历。

```

fn count_to(n: int) {
    var i: int = 0;
    while i < n {
        yield i;
        i = i + 1;
    }
}

for x in count_to(4) {
    print(x);
}

let g = count_to(2);
match g.next() {
    Option.Some(v) => { print(v); },
    Option.None() => {},
}

```

协议：next() -> Option<T> (Some 产出, None 结束)。

支持 while / if / elif / else / for (range 与列表) / break / continue, 以及**类方法生成器** (self 捕获为 __owner)。

```

fn filtered(n: int) {
    for i in range(n) {
        if i % 2 == 1 { continue; }
        if i > 4 { break; }
        yield i;
    }
}

class Counter {

```

```

    start: int;
    fn count(n: int) {
        var i: int = 0;
        while i < n {
            yield self.start + i;
            i = i + 1;
        }
    }
}
for x in Counter(10).count(3) { print(x); }

```

return; (无值) 可提前结束生成。

第 14 章 宏系统

14.1 心智模型

宏在**类型检查之前**展开为普通 Bolide AST。

调用必须带 !: `assert!(x)` 是宏, `assert(x)` 永远是函数。

层次:

1. **L0** 声明式模式宏 `macro name(...) quote { ... }`
2. **L1** `quote + $splice`
3. **L2** `comptime / comptime fn`
4. **L3** 属性 `@name / attr macro / @derive`

14.2 内置与自定义

```

assert!(x > 0);
assert_eq!(a, b);
let v = dbg!(1 + 2);
let s = stringify!(1 + 2);
todo!("later");

macro twice($x:expr) quote {
    ($x) + ($x);
}
print(twice!(21));

macro log_pair($a:expr, $b:expr) {
    print($a);
    print($b);
}

```

```
log_pair!(1, 2);

macro bind {
  ($name:ident = $val:expr) => {
    let $name = $val;
  },
}
bind!(n = 10);
```

14.3 导出与导入

```
// lib.bl
export macro add1($x:expr) quote { ($x) + 1; }

// main.bl
import "lib.bl" as lib;
print(add1!(41));
print(lib.add1!(9));
```

14.4 属性宏与 derive

```
@derive(Debug, Eq)
class Point {
  x: int;
  y: int;
}
let p: Point = Point(1, 2);
print(p.debug());
print(p.eq(Point(1, 2)));

@test
fn test_add() {
  assert!(1 + 1 == 2);
}

attr macro traced($item:item) {
  print("enter");
}
@traced
fn work() { print("body"); }
```

@derive(Debug, Eq, Clone, Default)、@getters 等可生成方法。

14.5 重复与 comptime

```
macro sum_all($x:expr $(, $rest:expr)*) quote {
  (fn() -> int {
    var __s = $x;
    $(
      __s = __s + $rest;
    )*
    return __s;
  })();
}
print(sum_all!(1, 2, 3, 4));

comptime fn fact(n: int) -> int {
  if n <= 1 { return 1; }
  return n * fact(n - 1);
}
let F: int = comptime { fact(5); };

```

调试展开:

```
bolide expand your_file.bl
```

设计细节见 docs/macro-design.md。标准宏集合: std/macros。

第 15 章 装饰器与上下文管理器 with

15.1 运行时装饰器

当 @name 不是内置/attr macro 时, 语义与 Python 相同: deco(f) -> f'。

```
fn logged(f: func() -> int) -> func() -> int {
  return fn() -> int {
    print("before");
    let r: int = f();
    print("after");
    return r;
  };
}

@logged
fn answer() -> int {
  return 42;
}
print(answer());

```

工厂装饰器:

```
fn repeat(n: int) -> func(func() -> int) -> func() -> int {
  return fn(f: func() -> int) -> func() -> int {

```

```

        return fn() -> int {
            var i: int = 0;
            var last: int = 0;
            while i < n {
                last = f();
                i = i + 1;
            }
            return last;
        };
    };
}
@repeat(3)
fn step() -> int { print("step"); return 1; }

```

多层 @a @b fn f 等价于 f = a(b(f))。

优先级: @test/@inline/@export → attr macro → 运行时装饰器。

15.2 with

```

class Resource {
    name: str;
    fn enter() -> str { print("enter"); return self.name; }
    fn exit() { print("exit"); }
}

with Resource("db") as r {
    print(r);
}
// with A() as x, B() as y { ... }

```

协议: enter() / exit() (finally 保证 exit)。

详见 docs/decorator-with-design.md。

第 16 章 枚举、模式匹配与错误处理

16.1 异常 try / catch / throw / finally

```

try {
    throw Error("boom");
} catch (e: Error) {
    print("caught: " + e.message);
} finally {
    print("cleanup");
}

```


- throw 只能抛 Error 或子类。
- catch (e: T) 支持子类匹配。
- finally 总会执行。
- throws 可作签名注解（工具元数据；尚未做 checked-exception 强制）。

自定义错误：

```
class MyError: Error {}
try {
    throw MyError("custom");
} catch (e: Error) {
    print(e.message);
}
```

实现：非 setjmp，而是显式 pending exception + catch 落点（JIT/AOT 跨函数均支持）。无 throw/try/? 的程序可跳过调用点 pending 检查（数值热路径收益大）。

16.2 Result / Option 与 ?

```
fn parse_number(text: str) -> Result<int, Error> {
    if text.len() == 0 {
        return Result.Err(Error("empty input"));
    }
    return Result.Ok(int(text));
}

fn load_value() -> Result<int, Error> {
    let value: int = parse_number("42")?;
    return Result.Ok(value + 1);
}
```

expr?:

- Ok(v) / Some(v) → v
- Err(e) / None → 早返回对应类型
- 错误类型不同时需 impl From<Src> for Dst

16.3 ! 与 try 表达式

```
// Result → 异常
let value: int = parse_number("42")!;

// 异常 → Result
let result: Result<int, Error> = try {
    let id: int = parse_number("42")!;
}
```

```
    id + 10;
};
```

16.4 实践建议

场景	选择
值可能缺失	<code>Option<T></code>
可恢复业务错误	<code>Result<T, E></code>
不可继续的失败	异常 / !
资源清理	<code>finally</code> 或 <code>with</code>

组合子见 `std/option`、`std/result`。

第 17 章 模块系统与包管理器

17.1 文件模块

```
// math_utils.bl
fn add(a: int, b: int) -> int {
    return a + b;
}

// main.bl
import "math_utils.bl";
print(math_utils.add(10, 20));

import "math_utils.bl" as mu;
print(mu.add(1, 2));
```

符号位于以模块名命名的命名空间，不污染全局。

17.2 路径解析顺序

1. 绝对路径
2. **导入方源文件目录**（相对路径）
3. `bolide.toml` 依赖
4. `BOLIDE_HOME`
5. `bolide` 可执行文件所在目录（发行版 `std/`）

不依赖进程当前工作目录。

标准库推荐**短路径**:

```
import "std/fs" as fs;
import "std/json" as json;
// 兼容旧路径 import "std/fs/fs.bl"
import "std/prelude" as std; // 常用模块集合
```

17.3 bolide.toml 与命令

```
[package]
name = "myapp"
version = "0.1.0"
description = "..."
authors = ["..."]
license = "MIT"
lib = "src/lib.bl"

[dependencies]
http = { git = "https://github.com/bolide-lang/http.git", ref = "v1.2.0" }
utils = { path = "../utils" }
db = { version = "0.3.0", registry = "https://registry.bolide.dev" }
```

```
bolide new myapp
bolide add ../utils --path
bolide add https://github.com/x/y.git --tag v1.0
bolide add http@1.2.0
bolide install
```

使用依赖:

```
import utils;
print(utils.greet());
import utils as u;
import "utils/extra.bl";
```

第 18 章 类型系统深入

18.1 静态类型 + 推断

```
let x = 1;           // int
let s = "hello";     // str
let xs = [1, 2];     // list<int>
```

复杂边界请显式标注:

```
let callbacks: list<func(int) -> int> = [add1, double];
let table: dict<str, list<int>> = {"a": [1, 2]};
```

18.2 类型一览

类型	说明
int / float / bool / str	标量
bigint / decimal	高精度
list<T> / dict<K,V> / 元组	容器
channel<T>	通道
func(...) -> R	函数值
Future<T>	冷协程
Task<T>	已启动任务
dynamic	动态装箱
dyn Trait	运行时多态接口
class / value / enum ADT	用户类型

18.3 dynamic 的边界

适合 JSON 边界、快速原型、异构配置。

不要把 dynamic 当默认业务类型；核心逻辑尽量静态化。

18.4 函数类型与 C ABI

用户业务：func(int) -> int。

FFI 的 C 函数指针同样写 func(...); fn 只用于声明与字面量。

C ABI 类型空间 (c_int、f64、*c_void...) 仅出现在 extern 中，见第 21 章。

第 19 章 内存管理：ARC、from、weak、unowned

函数参数的 **默认 / ref / owned** 与返回值 **from** 的「怎么写」见 第 6.3–6.4 节。

本章从**内存模型**角度说明：为什么需要它们、如何省 ARC、如何与 weak/unowned 配合。

19.1 ARC 默认模型

Bolide 默认用 **ARC (自动引用计数)** 管理堆对象。引用计数是**原子操作**（与 Swift / Rust Arc 同类内存序），跨线程 retain/release 不产生计数竞争。

多数业务代码**不必手写**任何内存关键字：

```
class Node {
    name: str;
}
let n = Node("root");
print(n.name);
// n 离开作用域 → 强引用归零 → 释放
```

ARC 的成本主要在：

- 赋值、传参、返回时可能的 **retain / release**
- 跨函数边界时成对的计数调整

对热路径上的大对象 (bigint、长 str、大 list、图节点)，减少无意义的 RC 流量会有可见收益——这正是 **默认借用**、**owned 移动**、**from 返回借用** 存在的原因。

19.2 传参如何影响 ARC

结合第 6.3 节，从 RC 视角再看三种模式：

模式	典型 RC 行为	一句话
默认 <code>x: T</code>	对 ARC 对象常传裸指针/借用， 函数体内只读时不额外 $\pm$ RC	最省事的只读接口
<code>ref x: T</code> <code>owned x: T</code>	传变量地址；赋值会按类型做释放旧值 / 绑定新值 移动 ：调用方释放绑定，被调方成为 owner	原地改调用方 避免「两边都以为自己拥有」

```
fn only_read(x: bigint) {
    print(x);           // 默认：借用式只读
}

fn bump(ref x: bigint) {
    x = x + 1B;         // 改的是调用方的绑定
}

fn take(owned x: bigint) {
    print(x);           // 结束后由本函数释放
}

var a: bigint = 1B;
only_read(a);
bump(a);
take(a);
// a 已 move
```

实践建议：

- API 默认用「默认模式」；只有需要改绑定或移交所有权时再升级。
- 不要「为了性能处处 owned」——可读性与调用约定成本更高。
- int/float 等标量按值传递，优先把精力放在大对象与容器上。

19.3 from: 返回借用并跳过 ARC

使用 from 声明**返回值生命周期依赖某参数**后，返回路径可以**不再对返回值做 retain**（借用，而非新的强引用）：

```
// README 同款: 跳过 ARC 开销
fn get_value(ref x: bigint) -> bigint from x {
    return x;
}

let a: bigint = 100B;
let b: bigint = get_value(a); // b 借用 a, 不增加引用计数
print(b);
```

编译期借用检查 (硬规则)

违反时**直接编译失败** (比运行期 UAF 安全得多):

1. **来源**: return 的值必须来自 from 所点名的参数。
2. **寿命**: 借用变量不能比来源活得更久 (悬空检测)。
3. **冻结来源**: 借用存活期间, 禁止对来源 **重新赋值 / 重声明** (否则旧对象释放, 借用变野)。
4. **禁止逃逸**: 借用值不能:
 - 存入 list / dict / 元组 / 对象字段
 - 经 push / insert 等进入容器
 - 经 channel 发送或作为 spawn 参数跨线程
 - 从未标注 from 的函数返回

需要逃逸时: **先拷贝再存储**, 例如 `bigint(b)`, 让新值重新进入正常 ARC 轨道。

和「普通返回」的对比

```
// 普通返回: 调用方通常拿到强引用 (可能 retain)
fn clone_like(x: bigint) -> bigint {
    return x; // 语义上是「给出可用的返回值」, 走常规所有权/RC
}

// from 返回: 明确「这是借用, 跟参数同寿」
fn view(ref x: bigint) -> bigint from x {
    return x;
}
```

from 不是语法糖装饰, 而是**契约**: 调用方必须遵守借用规则; 编译器帮你盯着。

与 ref 的配合

- from 经常和 ref 参数一起出现: 参数本身已是借用/可写引用, 返回再借出去。

- 也可以与只读默认参数组合（视类型与实现路径），但文档与示例以 `ref ... from` 最常见。
- **owned + from 同一参数**通常不合理：所有权已转移进函数，返回「借调用方」说不通。

性能直觉

操作	大致成本
热循环里每次普通返回 ARC 对象	可能反复 retain/release
from 返回借用	省掉该次返回路径上的 RC
value 类型字段	按值拷贝，不走堆 RC（见第 9 章）
inline fn	去掉调用约定开销（见第 6.6 节）

数值热路径优先：value + inline；对象图热路径再考虑 from / 减少临时 str 拼接。

19.4 weak / unowned

用于**打破循环引用**或表达「不延长寿命的观察指针」，与函数参数模式正交。

```
class Node {
    value: int;
}

let obj: Node = Node(42);

let w: weak Node = obj;      // 不增加引用计数
print(w.value);              // 访问前检查对象是否存活

let u: unowned Node = obj;    // 同样不增加引用；语义上假设 obj 更长寿
print(u.value);
```

对象已释放后再访问 weak/unowned → **确定性运行时错误并中止**（带诊断），**不是** use-after-free 未定义行为：

```
runtime error: weak/unowned reference accessed after object was deallocated
```

	weak	unowned
强 RC	不增加	不增加
语义意图	对象 可能 先死（如子 → 父回边）	对象 保证 比引用活得久
当前实现	访问时 trap 检查	同样 trap 检查

（未来若引入可选类型，weak 可能支持「已释放则空」分支；当前以 trap 保安全。）

19.5 概念总表

机制	出现位置	主要目的
默认参数	参数前无修饰	只读 / 常规传递, ARC 对象常免额外 retain
ref	参数前	修改调用方绑定
owned	参数前	移动所有权, 调用方失效
from	签名末尾 -> T from p	返回借用, 跳过返回路径 ARC
weak / unowned	类型前缀	不延长寿命的引用, 破坏 / 观察
value	类型声明	栈上/按值聚合, 避开堆 RC

19.6 并发与共享

- spawn 拒绝明显共享可变的 list/dict/bytes/dynamic
- 不可变 let 可传入; 容器会 clone/retain 副本
- **from 借用禁止作为 spawn 参数逃逸** (见上表)
- 共享可变状态优先 channel / std/atomic / std/sync

19.7 调试与学习建议

1. 先写对默认模式; 性能热点再 profile。
2. 需要改调用方时用 ref (可写 swap/increment 小练习)。
3. 需要「交出对象」时用 owned, 并确认调用点后不再使用原变量。
4. 需要「返回内部视图」时用 from, 并接受借用检查的约束。
5. 回归用例可参考仓库: tests/test_param_modes.bl、tests/test_borrow_checks_ok.bl、examples/lifetime_*.bl。

第 20 章 并发: 线程、通道、async/await

20.1 概念对照

概念	含义
async fn	返回冷 Future<T>, 不自动跑
await	等待 Future 或 Task
spawn	启动热 Task<T>
spawn thread	独立系统线程
pool(n)	限制块内 spawn 并发度
channel	线程安全消息
select	多路复用

20.2 async / await

```
async fn fetch_data(id: int) -> int {
    return id * 10;
}
let f1: Future<int> = fetch_data(1);
let r1: int = await f1;
```

20.3 spawn / pool

```
fn heavy_work(id: int) -> int {
    return id * id;
}

let t: Task<int> = spawn thread heavy_work(10);
let result: int = await t;

pool(4) {
    let t1: Task<int> = spawn heavy_work(1);
    let t2: Task<int> = spawn heavy_work(2);
    print(await t1);
    print(await t2);
}
```

20.4 spawn all / select

```
let results: (int, int) = spawn all {
    fetch_a(),
    fetch_b()
};

spawn select {
    res1 = task_fast() => { print("fast"); }
    res2 = task_slow() => { print("slow"); }
}
```

20.5 通道与 select

```
let ch: channel<int> = channel();

fn sender(c: channel<int>) {
    c.send(42);
}

spawn sender(ch);
let val: int = ch.recv();
```

```
select {
  val1 = ch1.recv() => { print("ch1"); }
  timeout(100) => { print("timeout"); }
  default => { print("none"); }
}
```

20.6 原子与同步

```
import "std/atomic" as atomic;
import "std/sync" as sync;

let counter: atomic.AtomicInt = atomic.new_int(0);
counter.add(1);

let lock: sync.Mutex = sync.mutex(10);
lock.add_int(5);
print(lock.get());
```

实践：少共享可变状态；优先消息传递；任务边界明确错误处理。

第 21 章 FFI 与 AOT / LLVM 发布

21.1 Bolide 调 C

```
extern "dyn:c" {
  fn abs(x: c_int) -> c_int;
}
extern "dyn:m" {
  fn sqrt(x: f64) -> f64;
}
let a: int = abs(-42);
let b: float = sqrt(16.0);
```

库标识

标识	JIT	AOT	说明
dyn:name	动态加载	动态加载	可移植逻辑名
lib:name	不支持	链接	静态/导入库
auto:name	同 dyn	同 lib	单源码双模式
bolide	runtime	runtime	标准库内部

不要写 xxx.dll / libxxx.so 进源码。
dyn:c / dyn:m 映射各平台 C 运行时与数学库。

C ABI 类型

类别	写法
平台整数	c_int, c_size_t, ...
固定宽度	i32, u64, f64, ...
指针	*T, *c_void
函数指针	func(T...) -> R

Bolide int (64 位) $\neq$ C int。业务 API 用语言类型；仅 raw extern 用 C ABI。

21.2 C 调 Bolide

```
export fn add(a: int, b: int) -> int { return a + b; }
export fn scale(x: float, k: float) -> float { return x * k; }
```

```
bolide compile mathlib.bl --lib --header
# mathlib.Lib + mathlib.h (Unix: Libmathlib.a)
```

C 侧链接 mathlib + bolide_runtime。跨边界优先数值/指针签名。

21.3 AOT 发布清单

```
bolide compile main.bl -o app
bolide compile main.bl -o app --backend llvm # 可选
```

检查项：

1. 依赖的动态库是否在目标机可解析
2. 数据文件 / 模板相对路径是否相对可执行文件合理
3. GUI / Web 是否已在本机 AOT 跑通
4. 异常路径与错误信息是否友好

21.4 性能后端选择

后端	优点	注意
Craneflight	默认完整、编译快	峰值略逊 LLVM
LLVM	数值/循环常接近 C	需 clang；部分高级特性覆盖以仓库文档为准

list[i] / len 等保持安全语义；热路径可用 reserve/resize、value、inline fn、减少异常路径。

第 22 章 标准库全景

索引: std/README.md; 教程: docs/standard-library.md。

22.1 导入约定

```
import "std/fs" as fs;
import "std/prelude" as std;
```

22.2 模块地图

核心配套: option result traits macros assert prelude

数据与算法: collections iter sort math random hash vec3 cache

文本: text buffer bytes encoding json csv regex template html table

系统: fs path io env process time log cli

并发: atomic sync arena

网络与应用: http web url crawler db sqlite gui uuid config

22.3 选型速查

目标	模块
CLI 工具	cli fs path log env
爬虫 / API	http crawler html url cache
Web 应用	web json template sqlite/db
数据处理	csv regex text table json
桌面 GUI	gui
并发结构	atomic sync channel (语言内建)

22.4 失败语义

系统/IO 类 API 多数返回 bool 表示成败; 需要结构化错误时用 Result 包装或查模块文档。std/process 返回 ProcessResult (避免与 ADT Result 混淆)。

22.5 实现分层 (进阶)

1. **Rust runtime** (extern "bolide") —与对象模型紧密的能力
2. **独立静态标准库** (extern "std:name")
3. **外部 C FFI** (lib: / dyn:)

用户只依赖稳定 .bl API。

22.6 核心模块用法精讲

CLI

```
import "std/cli" as cli;

let specs: list<cli.Spec> = [
    cli.help_flag(),
    cli.option("file", "f", "PATH", "input.txt", "input file"),
    cli.flag("verbose", "v", "verbose output"),
    cli.required_option("name", "n", "NAME", "project name"),
];
let args: cli.Args = cli.parse_or_exit(specs, "demo tool");
print(args.value("file"));
print(str(args.flag("verbose")));
```

支持长/短选项、粘连短选项（如 -fin.bl）、默认值、必填、位置参数与帮助文本。

文件系统与路径

```
import "std/fs" as fs;
import "std/path" as path;

let p: str = path.join("data", "app.json");
if fs.exists(p) {
    let text: str = fs.read_to_string(p);
    print(text);
}
fs.write_string("out.txt", "hello\n");
```

JSON

```
import "std/json" as json;

let v = json.parse("{\"n\": 1, \"items\": [true, \"x\"]}");
print(json.stringify_pretty(v, 2));
// 对象 → dict<str, dynamic>; 数组 → list<dynamic>
// get_path(v, "a.b.0"); 失败时 parse 返回 null, 可用 parse_error()
```

时间 / 日志 / 环境

```
import "std/time" as time;
import "std/log" as log;
import "std/env" as env;

log.info("boot");
let t0: int = time.now_ms();
```

```
time.sleep_ms(10);
print(time.now_ms() - t0);
print(env.get_or("HOME", ""));
```

正则 / 文本 / CSV

```
import "std/regex" as regex;
import "std/text" as text;
import "std/csv" as csv;

let re = regex.compile(r"\d+");
print(re.is_match("ab12cd"));

let rows: list<list<str>> = csv.parse("a,b\n1,2\n");
print(csv.stringify(rows));
```

集合与迭代工具

```
import "std/collections" as col;
import "std/iter" as iter;

// Set / Queue / Stack / Deque / Counter / 优先队列
// iter: take / drop / zip / unique / sum / chunk ...
```

原子与同步

见第 20 章; 完整 API 见 docs/standard-library.md。

第 23 章 Web、模板与数据库

23.1 最小 Web 服务

```
import "std/web" as web;

fn index(req: web.Request) -> web.Response {
    return web.html("<h1>Hello Bolide</h1><p>path=" + req.path() + "</p>");
}

fn hello(req: web.Request) -> web.Response {
    return web.text("hello " + req.path_param("name"));
}

fn api_echo(req: web.Request) -> web.Response {
    return web.json("{\"path\":\"" + req.path() + "\"}");
}
```

```

}

let app: web.App = web.app();
app.get("/", index);
app.get_async("/hello/{name}", hello);
app.post("/api/echo", api_echo);
app.static_files("/static", "public");
// app.set_workers(4); // 压测或部署调优
app.listen(8080); // 或 app.run("127.0.0.1", 8080)

```

能力清单

类别	内容
方法	GET/POST/PUT/PATCH/DELETE/HEAD/OPTIONS/TRACE/CONNECT
路由	精确路径、/posts/{id}、静态目录
请求	method、path、query、header、cookie、form、body 文本/bytes
响应	text/html/json/bytes/empty/redirect; 自定义 status/header/cookie
会话	get/set/contains/remove/clear/destroy/regenerate
连接	HTTP/1.1 keep-alive、Content-Length、HEAD 无 body、405/OPTIONS

示例入口

路径	说明
examples/web_hello.bl	Hello 服务
examples/web_sse.bl	Server-Sent Events
examples/web_ws_echo.bl	WebSocket 回显
examples/web_upload.bl	上传
examples/blog/	完整博客
examples/chat/	聊天

```

bolide run examples/blog/main.bl
bolide compile examples/blog/main.bl -o examples/blog/main.exe

```

本机参考：hello 服务可达约 **150k+ RPS**（结果随硬件变化；压测脚本见仓库 README）。

23.2 HTTP 客户端

```
import "std/http" as http;
```

```
// 常见: get / post_json; 可设 timeout
// 请求层错误 (DNS、连接、TLS、超时、非法 URL) 不再静默吞掉
// 通过 Response.error / ClientResponse.error() 读取
```

适合写爬虫、调用 REST API、健康检查脚本。与 std/url、std/html、std/crawler 组合可搭轻量抓取管线。

23.3 模板引擎

```
import "std/template" as template;

let html: str = template.render(
    "<h1>{{ title }}</h1>{% if show %}{% for post in posts %}<p>{{ post.title }}</p>{
    {
        "title": "Blog",
        "show": true,
        "posts": [{ "title": "A" }, { "title": "B" } ],
    }
);
```

语法	含义
{{ expr }}	HTML 转义输出
{!! expr !!}	原始 HTML (仅可信内容)
{% if %}...{% else %}...{% endif %}	条件
{% for x in xs %}...{% endfor %}	遍历
post.title	点路径读 dict/对象字段

23.4 文件数据库 std/db

```
import "std/db" as db;

let database: db.Database = db.open("data/blog");
database.create_table("posts", "title,slug,body,published");
let id: int = database.insert("posts", {
    "title": "Hello",
    "slug": "hello",
    "body": "text",
    "published": true,
});
let row: dict<str, dynamic> = database.get("posts", id);
let posts: list<dict<str, dynamic>> = database.all("posts");
let pub = database.where_eq("posts", "published", true);
print(database.count("posts"));
database.close();
```

API: create_table / insert / update / delete / get / all / where_eq / count / last_error.

数据以目录为库、表为文件，适合示例与中小应用。

23.5 SQLite

需要 SQL 与更强查询时用 std/sqlite (见标准库文档与模块源码)。博客示例当前主路径是 std/db + 模板。

23.6 把三块拼成应用

1. web 接路由与 session
2. db/sqlite 持久化
3. template 渲染 HTML
4. JIT 开发 → AOT 发布单文件

完整可运行参考：examples/blog/ (文章、评论、登录、后台)。

第 24 章 GUI 开发

24.1 模型

后端：**egui/eframe** (在 runtime 中)。

模式：应用状态放在 var 全局或对象里；gui.run(title, w, h, root) 每帧调用 root(ui) 声明式绘制。

用户层类型：gui.Ui、str、int、bool、func(gui.Ui)。

24.2 最小窗口

```
import "std/gui" as gui;

var count: int = 0;
var status: str = " 准备";

fn toolbar(ui: gui.Ui) {
    if ui.button(" 增加") {
        count = count + 1;
        status = " 计数 " + str(count);
    }
    if ui.button(" 清零") {
        count = 0;
        status = " 已清零";
    }
}

fn body(ui: gui.Ui) {
```

```

    ui.heading("Bolide GUI");
    ui.pack_left(8, toolbar);
    ui.space(12);
    ui.strong(status);
}

fn root(ui: gui.Ui) {
    ui.pad(16, 16, body);
}

gui.run("Bolide GUI", 420, 280, root);
// gui.run_default(...) 也可用

```

24.3 布局 API

API	作用
pad / space	内边距与间距
pack_top/left/right/bottom	方向排列
row / column	水平 / 垂直
grid(id, columns, striped, child)	表格布局
fill / fill_width / fill_height	占满可用空间
left / right / centered / align	对齐
frame / scroll / indent / collapsing	容器
width / height / size / place	固定尺寸或绝对位置

24.4 常用控件

- 文本: label、heading、small、strong
- 命令: button、selectable、link
- 输入: text_input、password_input、multiline_input、checkbox、slider
- 状态: progress / progress_pct、separator
- 查询: available_width()、available_height()
- 重绘: request_repaint()

24.5 示例与发布

文件	内容
examples/calculator.bl	计算器: fill + grid 按键区
examples/gui_showcase.bl	控件展示
examples/gui_project_dashboard.bl	仪表盘式布局

文件	内容
examples/starfield.bl	动画/重绘

```
bolide run examples/calculator.bl
bolide compile examples/calculator.bl -o examples/calculator.exe
```

AOT 时 GUI 后端随 runtime 静态链接；Windows 下已适配 AOT 入口线程的事件循环。中文显示依赖系统 CJK 字体回退。

第 25 章 报错诊断、调试与测试

25.1 读懂编译错误

关注：文件路径、行列、源码片段、caret、help。
导入模块内错误会指向**真实源文件与函数名**，而不是仅 import 行。

典型未定义名：

```
Error: Compile error: Undefined variable or function: missing_name
--> example.bl:1:9
help: Define the name before using it, or check for a spelling/import mistake.
```

25.2 调试手段

1. print / f-string 与 dbg!(expr)
2. bolide expand file.bl 查看宏/装饰器脱糖
3. 先 JIT 缩小复现，再 AOT
4. 对比 --backend cranelift 与 llvm
5. 对 FFI：单独测 C 侧与 Bolide 侧
6. 对并发：先单线程逻辑正确，再加 spawn

25.3 轻量测试

```
import "std/assert" as assert;

assert.reset();
assert.equal("sum", 1 + 2, 3);
assert.is_true("contains", "bolide".contains("lid"));
assert.contains("msg", "hello bolide", "bolide");
print(assert.summary());
// passed_count / failed_count / ok
```

也可用 `@test + assert!`:

```
@test
fn test_add() {
    assert!(1 + 1 == 2);
}
```

仓库 tests/ 覆盖宏、装饰器、生成器、trait、运算符、标准库等回归。

25.4 常见坑

现象	可能原因
let 不能 push	应用 var
宏不展开	忘了！
dynamic 与 dyn Trait 混淆	完全不同概念
顶层 map 类型怪	补结果类型注解
AOT 缺库	dyn: 目标机找不到 DLL/so
weak 访问直接中止	对象已释放（确定性 trap, 非 UAF）
链式比较	a < b < c 语法错误，请拆开写
lib: 在 JIT	JIT 无链接阶段，开发期用 dyn: 或 auto:

第 26 章 性能优化与工程实践

26.1 优化优先级

1. 算法与数据结构（最大收益）
2. 减少分配与字符串拼接（std/buffer）
3. list.reserve / resize; 避免反复扩容
4. 热路径 value + inline fn
5. 无 throw/try/? 的路径可跳过 pending 检查
6. list.len() 已内联；常用 math 可降为机器指令
7. 数值内核尝试 --backend llvm
8. 并发：pool + channel; 少共享可变状态

26.2 安全与速度的边界

- list[i] 始终边界检查（越界读 0、越界写忽略）

- 不以去掉检查换 bench 数字
- 需要批量初始化时用 `resize/reserve`

26.3 工程约定

- 模块边界清晰；公共 API 放 `lib.bl`
- 标准库一律短路径 `import "std/xxx"`
- 业务错误 `Result`，意外失败才异常
- 发布前：JIT 正确性 → AOT 冒烟 → 关键路径测试
- `bolide.lock` 入库；依赖版本钉死
- 示例与 `tests/` 同步演进
- 文档与 `CHANGELOG.md` 随版本更新

26.4 基准与对照

仓库 `bench/`：

程序	说明
<code>fib</code>	递归整数
<code>sieve</code>	筛法
<code>mandelbrot</code>	浮点密集
<code>nbody</code>	多体模拟

与 `C clang -O3 -march=native` 对照；脚本见 `bench/README.md`、`bench/run_all.ps1`。
HTTP 压测见 `README Web` 章节。

26.5 发布检查清单

1. `bolide compile main.bl -o app` 成功
2. 在干净目录运行（无开发机 `BOLIDE_HOME` 幻觉）
3. 模板/数据相对路径正确
4. `dyn`：依赖在目标机可解析
5. GUI/Web 冒烟
6. 错误路径信息可读

第 27 章 综合项目实战

27.1 项目 A：命令行任务管理器（可落地骨架）

目标：本地 JSON 存储的 todo CLI (add / list / done)。

```
todo/  
  bolide.toml  
  src/  
    main.bl  
  data/  
    tasks.json  # 运行时生成
```

核心思路：

```
import "std/cli" as cli;  
import "std/fs" as fs;  
import "std/json" as json;  
import "std/table" as table;  
  
// 1) 解析子命令与参数 (cli.parse_or_exit)  
// 2) 若无 tasks.json 则写 "[]"  
// 3) json.parse 读成 list<dynamic>  
// 4) add: push 一条 dict{id,title,done}  
// 5) list: table 打印  
// 6) done: 按 id 改 done=true 再 stringify 写回  
// 7) bolide compile src/main.bl -o todo
```

扩展练习：

- std/assert 覆盖 JSON 读写
- 用 std/uuid 生成 id
- 用 std/log 记录操作

27.2 项目 B：迷你博客

直接阅读并改造 examples/blog/：

1. 路由表：列表 / 详情 / 登录 / 后台
2. std/db 表：posts / users / comments
3. std/template + templates/*.html
4. session 鉴权
5. JIT 开发，AOT 发布

改造练习：加标签过滤、RSS、Markdown 字段、SQLite 迁移。

27.3 项目 C: GUI 计算器

阅读 `examples/calculator.bl`:

- 上方显示区固定高度
- 底部 `fill + grid(4, ...)` 按键
- 状态: 当前输入、运算符、累加器

练习: 历史记录列表、% 运算、键盘事件、暗色主题。

27.4 项目 D: 数值小引擎

参考 `examples/raytracer_vt.bl`、`examples/parallel_mandelbrot.bl`:

- `value Vec3 + inline fn`
- 可选 `pool` 并行扫描线
- `--backend llvm` 对照

练习: 景深、简单材质、`std/vec3`。

27.5 项目 E: 导出 C 库

```
export fn fib(n: int) -> int {
  if n < 2 { return n; }
  return fib(n - 1) + fib(n - 2);
}

export fn clamp_i(x: int, lo: int, hi: int) -> int {
  if x < lo { return lo; }
  if x > hi { return hi; }
  return x;
}
```

```
bolide compile mathlib.bl --lib --header
# Windows: cl main.c mathlib.lib bolide_runtime.lib
# Linux:   cc main.c libmathlib.a libbolide_runtime.a
```

记住: 跨 C 边界优先数值/指针签名。

27.6 项目 F: 特性总览程序

运行并阅读:

```
bolide run examples/feature_showcase.bl
bolide run examples/neon_lang.bl
```

把其中感兴趣的片段拆成自己的小库。

第 28 章 附录

附录 A • 语法速查

```
// 绑定
let x: int = 1;
var y: int = 2;

// 函数与传参
fn f(a: int = 0, *xs: int, **kw: int) -> int { return a; }
fn only_read(x: bigint) { print(x); }           // 默认
fn bump(ref x: bigint) { x = x + 1B; }          // ref
fn take(owned x: bigint) { print(x); }          // owned
fn view(ref x: bigint) -> bigint from x { return x; } // from: 返回借用, 省 ARC
inline fn g(x: int) -> int { return x; }
export fn h(x: int) -> int { return x; }

// 类型
value V { a: int; }
class C { a: int; fn m() {} }
trait T { fn m(); }
impl T for C { fn m() {} }

// 控制
if / elif / else / while / for / break / continue
match / if let / while let

// 错误
try { } catch (e: Error) { } finally { }
Result.Ok / Result.Err / Option.Some / Option.None
expr? expr!

// 元编程
macro m($x:expr) quote { $x }
name!(...)
@derive(Debug) @logged
with r() as x { }

// 并发
spawn f(); await t; pool(n) { } channel(); select { }

// 模块
import "std/fs" as fs;
```


附录 B • 命令速查

```
bolide run FILE.bl
bolide run FILE.bl --backend llvm
bolide compile FILE.bl -o OUT
bolide compile FILE.bl --lib --header
bolide expand FILE.bl
bolide new NAME
bolide add SPEC
bolide install
```

附录 C • 学习资源

资源	路径
本书 Markdown	docs/book/bolide-from-zero-to-mastery.md
标准库教程	docs/standard-library.md
宏设计	docs/macro-design.md
装饰器设计	docs/decorator-with-design.md
标准库索引	std/README.md
示例	examples/
变更日志	CHANGELOG.md
英文 README	README_EN.md

附录 D • 版本与许可

- 本书基准: **Bolide 0.14.1**
- 语言与工具链以仓库为准
- 许可证: **MIT**

结束语

Bolide 仍在快速演进——以仓库文档与测试为准，把本书当作**可运行的地图**，而不是永恒的规格书。
祝你写得开心，跑得飞快。